



中国科学技术大学
University of Science and Technology of China

自上而下的解析与翻译

《编译原理和技术(H)》

张昱

yuzhang@ustc.edu.cn

中国科学技术大学
计算机科学与技术学院



自上而下解析器的生成

- ☐ 递归下降的预测解析
- ☐ 非递归的预测解析（预测分析表）
- ☐ 错误恢复



预测解析器

□ 预测解析

- 根据面临的**输入符号**，预测用非终结符的**哪个选择**进行展开(推导)

□ 递归下降(recursive-descent)的预测解析

- 为每一个非终结符写一个解析函数
- 这些函数可能是递归的

递归：解析函数可能是递归的
下降：自上而下

例 下面的文法产生Pascal语言的类型子集

```
type → simple  
      | ↑ id  
      | array [simple] of type  
simple → integer  
      | char  
      | num dotdot num
```



递归下降的预测解析器

type $\rightarrow$ *simple* | $\uparrow$ *id* | *array* [*simple*] *of type*
simple $\rightarrow$ *integer* | *char* | *num* *dotdot* *num*

```
void match (terminal t) {  
    if (lookahead == t) lookahead = nextToken();  
    else error();  
}  
  
void type( ) {  
    if ( (lookahead == integer) || (lookahead == char) || (lookahead == num) )  
        simple();  
    else if ( lookahead == '↑' ) { match('↑'); match(id); }  
    else if (lookahead == array) {  
        match(array); match( '[' ); simple();  
        match( ']' ); match(of ); type();  
    }  
    else error();  
}
```



递归下降的预测解析器

type $\rightarrow$ *simple* | $\uparrow$ id | array [*simple*] of *type*
simple $\rightarrow$ integer | char | num dotdot num

```
void simple( ) {  
    if ( lookahead == integer) match(integer);  
    else if (lookahead == char) match(char);  
    else if (lookahead == num) {  
        match(num); match(dotdot); match(num);  
    }  
    else error( );  
}
```

递归下降的预测解析程序的构造 与 文法结构 直接对应
→ 编程简单



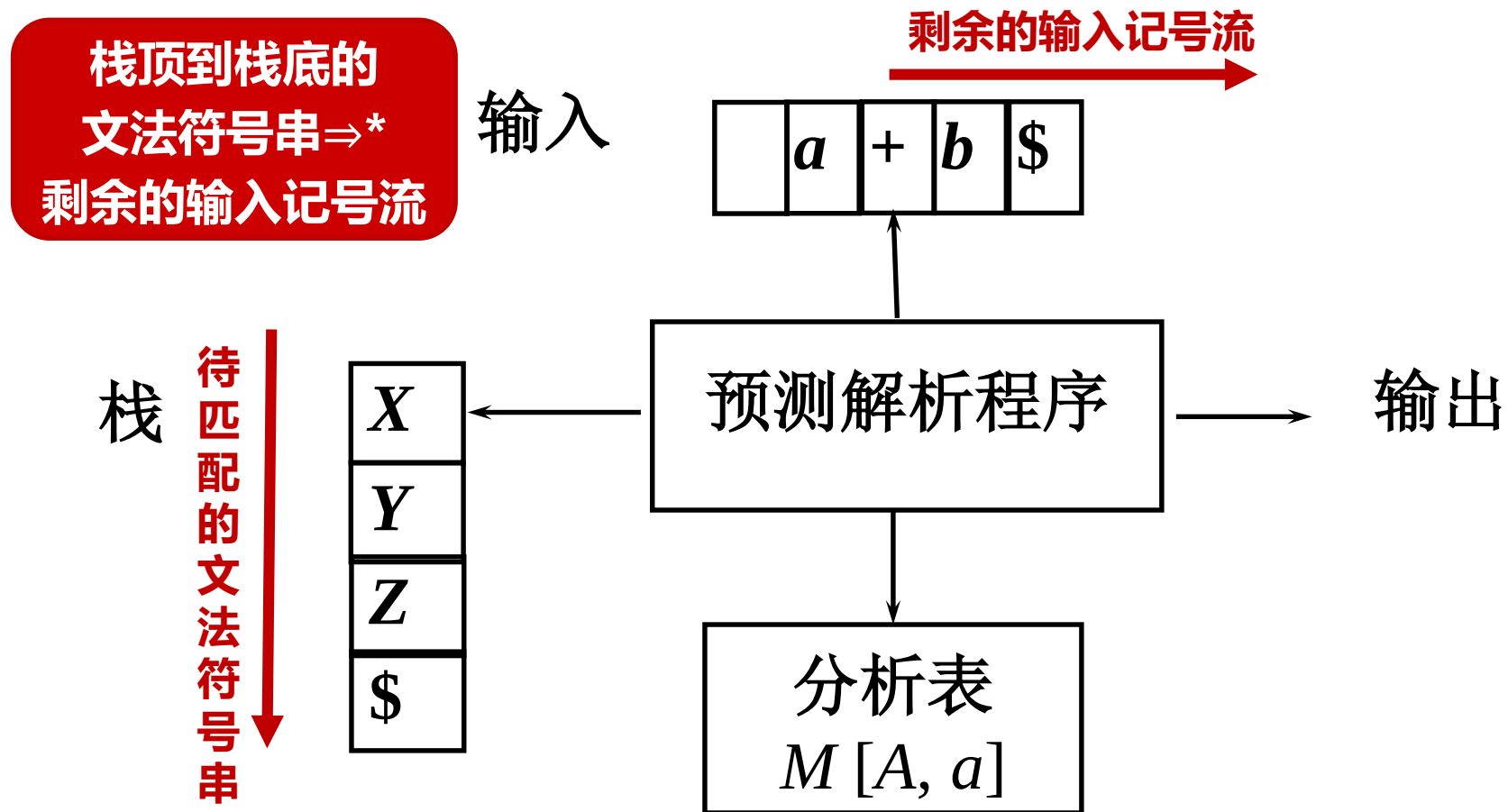
自上而下解析器的生成

- 递归下降的预测解析
- 非递归的预测解析（预测分析表）
- 错误恢复



非递归的预测解析

Nonrecursive Predictive Parsing





非递归的预测解析算法

让输入指针 ip 指向剩余输入记号流 w \$的符号 a ;

令 X 等于栈顶符号;

while ($X \neq \$$) { /* 栈非空 */

if (X 是 a) 把 X 从栈顶弹出并把 ip 推进到指向下一个符号;

else if (X 是终结符) **error** ();

else if ($M[X, a]$ 是出错入口) **error** ();

else if ($M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$) {

 输出产生式 $X \rightarrow Y_1 Y_2 \dots Y_k$;

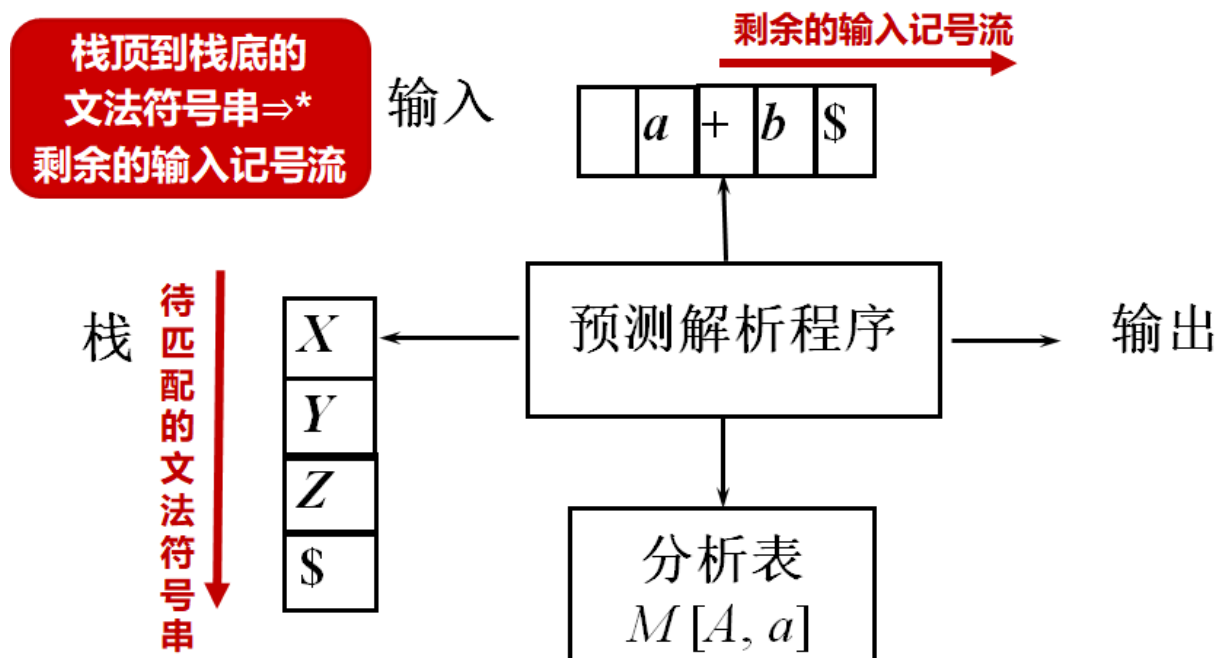
 从栈中弹出 X ;

 把 $Y_k, Y_{k-1}, \dots, Y_1$ 依次压入栈, Y_1 在栈顶;

 }

 令 X 等于栈顶符号;

}





预测分析表

- 行：非终结符；列：终结符或\$；单元：产生式
- 例题7-1 预测解析器的预测分析表如下，请给出对 $\text{id} * \text{id} + \text{id}$ 的解析

非终结符	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		



预测解析举例

预测解析器接受输入 $\text{id} * \text{id} + \text{id}$
的前一部分动作

非终结符	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

栈	输 入	输 出
$\$E$	$\text{id} * \text{id} + \text{id}\$$	



预测解析举例

预测解析器接受输入 $id * id + id$ 的前一部分动作

栈	输 入	输 出
$\$E$	$id * id + id\$$	
$\$E'T$	$id * id + id\$$	$E \rightarrow TE'$



预测解析举例

预测解析器接受输入 $id * id + id$ 的前一部分动作

栈	输 入	输 出
$\$E$	$id * id + id\$$	
$\$E'T$	$id * id + id\$$	$E \rightarrow TE'$
$\$E'T'F$	$id * id + id\$$	$T \rightarrow FT'$



预测解析举例

预测解析器接受输入 $id * id + id$ 的前一部分动作

栈	输 入	输 出
$\$E$	$id * id + id\$$	
$\$E 'T$	$id * id + id\$$	$E \rightarrow TE '$
$\$E 'T 'F$	$id * id + id\$$	$T \rightarrow FT '$
$\$E 'T ' id$	$id * id + id\$$	$F \rightarrow id$



预测解析举例

预测解析器接受输入 $id * id + id$ 的前一部分动作

栈	输 入	输 出
$\$E$	$id * id + id\$$	
$\$E 'T$	$id * id + id\$$	$E \rightarrow TE '$
$\$E 'T 'F$	$id * id + id\$$	$T \rightarrow FT '$
$\$E 'T 'id$	$id * id + id\$$	$F \rightarrow id$
$\$E 'T '$	$* id + id\$$	



预测解析举例

预测解析器接受输入 $id * id + id$ 的前一部分动作

栈	输 入	输 出
$\$E$	$id * id + id\$$	
$\$E 'T$	$id * id + id\$$	$E \rightarrow TE '$
$\$E 'T 'F$	$id * id + id\$$	$T \rightarrow FT '$
$\$E 'T 'id$	$id * id + id\$$	$F \rightarrow id$
$\$E 'T '$	$* id + id\$$	
$\$E 'T 'F *$	$* id + id\$$	$T' \rightarrow *FT '$



预测解析举例

预测解析器接受输入 $id * id + id$ 的前一部分动作

栈	输 入	输 出
$\$E$	$id * id + id\$$	
$\$E 'T$	$id * id + id\$$	$E \rightarrow TE '$
$\$E 'T 'F$	$id * id + id\$$	$T \rightarrow FT '$
$\$E 'T 'id$	$id * id + id\$$	$F \rightarrow id$
$\$E 'T '$	$* id + id\$$	
$\$E 'T 'F *$	$* id + id\$$	$T' \rightarrow *FT '$
$\$E 'T 'F$	$id + id\$$	



预测解析举例

预测解析器接受输入 $id * id + id$ 的前一部分动作

栈	输 入	输 出
$\$E$	$id * id + id\$$	
$\$E 'T$	$id * id + id\$$	$E \rightarrow TE '$
$\$E 'T 'F$	$id * id + id\$$	$T \rightarrow FT '$
$\$E 'T ' id$	$id * id + id\$$	$F \rightarrow id$
$\$E 'T '$	$* id + id\$$	
$\$E 'T 'F *$	$* id + id\$$	$T' \rightarrow *FT '$
$\$E 'T 'F$	$id + id\$$	
$\$E 'T ' id$	$id + id\$$	$F \rightarrow id$



预测解析举例

预测解析器接受输入id * id + id的前一部

栈	输入	动作
\$E	id * id + id\$	
\$TE'	id * id + id\$	输出E → TE'
\$FT'E'	id * id + id\$	输出T → FT'
\$id T'E'	id * id + id\$	输出F → id
\$T'E'	* id + id\$	匹配id
\$* FT'E'	* id + id\$	输出T' → *FT'
\$FT'E'	id + id\$	匹配*
\$id T'E'	id + id\$	输出F → id
\$T'E'	+ id\$	匹配id
\$E'	+ id\$	输出T' → ε
\$+ TE'	+ id\$	输出E' → +TE'
\$TE'	id\$	匹配+
\$FT'E'	id\$	输出T → FT'
\$id T'E'	id\$	输出F → id
\$T'E'	\$	匹配id
\$E'	\$	输出T' → ε
\$	\$	输出E' → ε

非终结符	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		



预测分析表的构造

□ 预测分析表(predictive parsing table)

行：非终结符；列：终结符 或\$；单元：产生式

$M[A, a]$ 产生式 $A \rightarrow \alpha$ 表示在面临 a 时，将栈顶符号 A 替换为 α

□ 构造方法

- (1) 对文法的每个产生式 $A \rightarrow \alpha$ ，执行(2)和(3)
- (2) 对 $\text{FIRST}(\alpha)$ 的每个终结符 a ，把 $A \rightarrow \alpha$ 加入 $M[A, a]$
- (3) 如果 ε 在 $\text{FIRST}(\alpha)$ 中，对 $\text{FOLLOW}(A)$ 的每个终结符 b （包括\$），把 $A \rightarrow \alpha$ 加入 $M[A, b]$
- (4) M 中其它没有定义的条目都是error



多重定义

例题7-2 $stmt \rightarrow \text{if } expr \text{ then } stmt \ e_part \mid \text{other}$
 $e_part \rightarrow \text{else } stmt \mid \varepsilon$ $expr \rightarrow b$

非终结符	输入符号					
	other	<i>b</i>	else	if	then	\$
<i>stmt</i>	$stmt \rightarrow \text{other}$			$stmt \rightarrow \text{if...}$		
<i>e_part</i>			$e_part \rightarrow \text{else } stmt$ $e_part \rightarrow \varepsilon$			$e_part \rightarrow \varepsilon$
<i>expr</i>		$expr \rightarrow b$				

多重定义条目意味着文法左递归或者是二义的



多重定义的消除

删去 $e_part \rightarrow \varepsilon$ ，这正好满足 **else**和最近的**then**配对

LL(1)文法 $\Leftrightarrow$ 预测分析表无多重定义的条目

非终结符	输入符号					
	other	b	else	if	then	\$
$stmt$	$stmt \rightarrow \mathbf{other}$			$stmt \rightarrow \mathbf{if...}$		
e_part			$e_part \rightarrow \mathbf{else\ stmt}$ $e_part \rightarrow \varepsilon$			$e_part \rightarrow \varepsilon$
$expr$		$expr \rightarrow b$				

没有通用的方法来指导多重定义条目的删除



自上而下解析器的生成

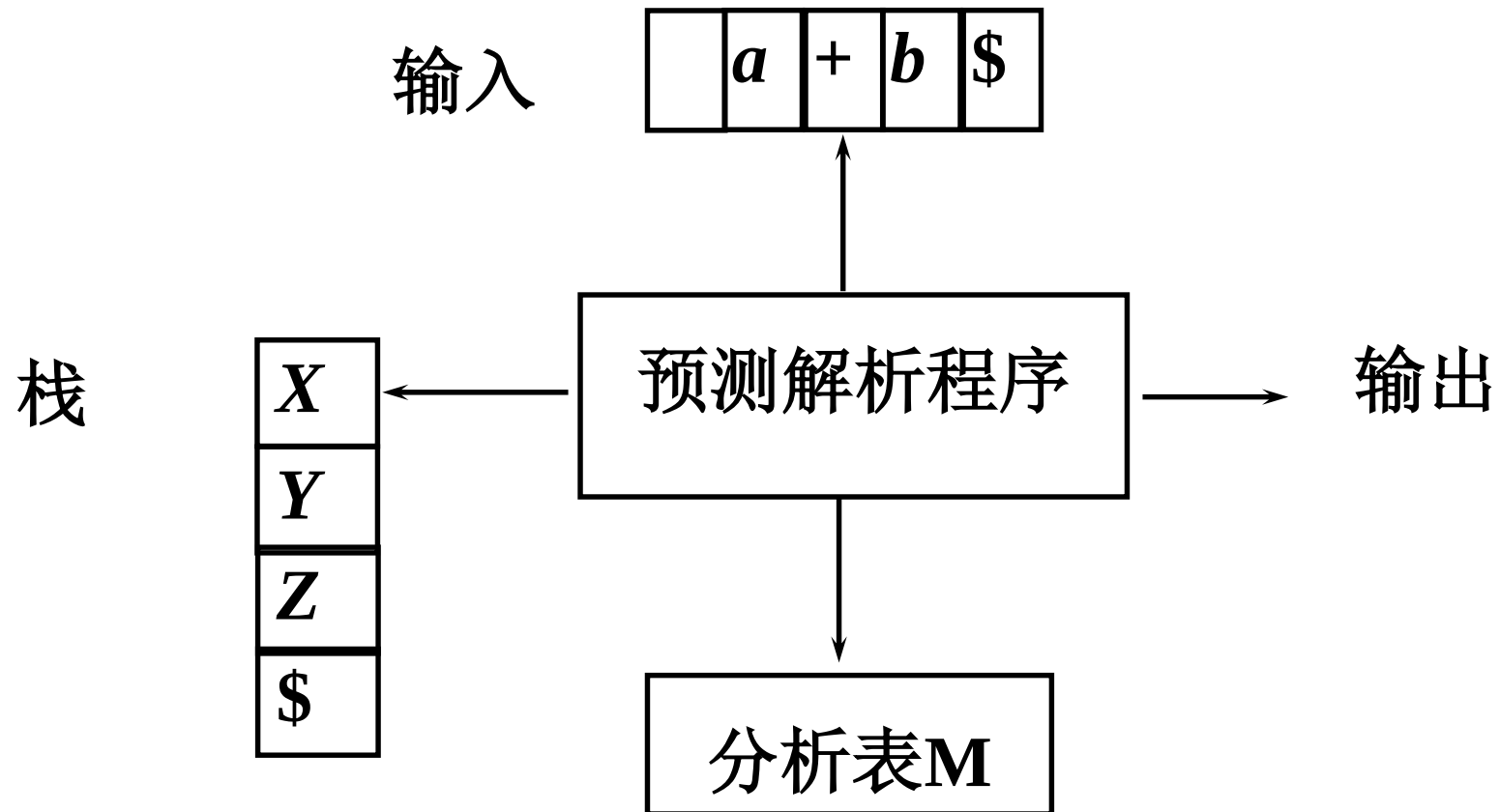
- ☐ 递归下降的预测解析
- ☐ 非递归的预测解析（预测分析表）
- ☐ 错误恢复



预测解析的错误恢复

□ 非递归预测解析在什么场合下发现错误

- 栈顶的终结符和下一个输入符号不匹配

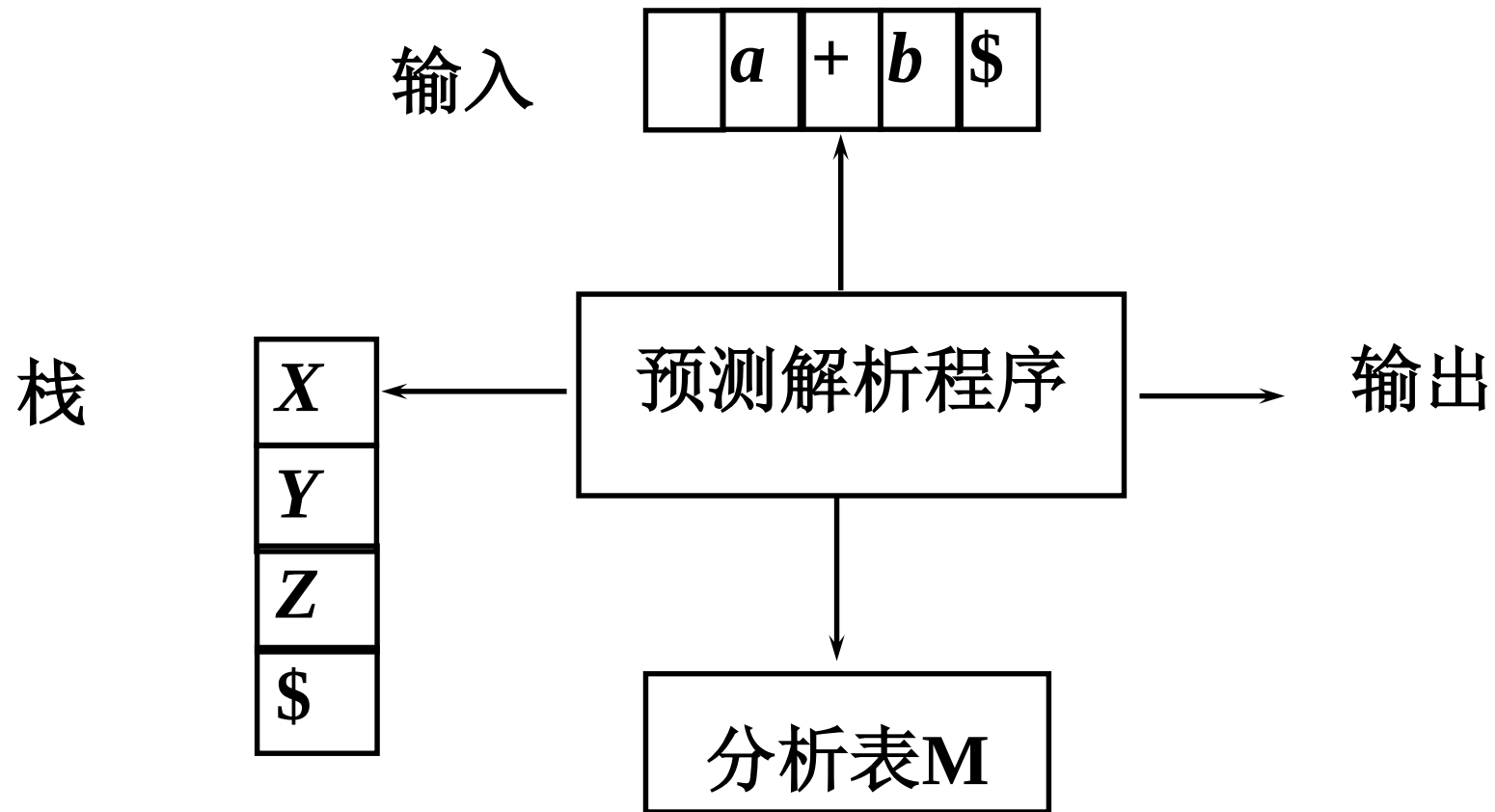




预测解析的错误恢复

□ 非递归预测解析在什么场合下发现错误

- 栈顶是非终结符 A ，输入符号是 a ，而 $M[A, a]$ 是空白





预测解析的错误恢复

□ 非递归预测解析

采用紧急方式(panic mode)的错误恢复

- 发现错误时，抛弃输入记号直到其属于某个指定的同步记号(synchronizing tokens)集合为止

□ 同步(synchronizing)

- 同步：词法分析器当前提供的记号流能够构成的语法构造，正是语法分析器所期望的
- 不同步的例子
语法分析器期望剩余的前缀构成过程调用语句，而实际剩余的前缀形成的是赋值语句



预测解析的错误恢复

□ 同步记号集合的选择

- 把FOLLOW(A)的所有终结符放入非终结符A的同步记号集合

if *expr* then *stmt* 同步记号

出错

(then和分号等记号是*expr*的同步记号)

- 把高层构造的首终结符加到低层构造的同步记号集中

a = *expr*; if ...

出错

同步记号

(语句的首终结符作为表达式的同步记号, 以免表达式出错又遗漏分号时忽略if语句等一大段程序)



预测解析的错误恢复

□ 同步记号集合的选择

- 把FOLLOW(A)的所有终结符放入非终结符A的同步记号集合
- 把高层构造的首终结符加到低层构造的同步记号集中
- 把FIRST(A)的终结符加入A的同步记号集合

$a = \text{expr};$ **if** ...

出错

同步记号

(语句的开始符号作为语句的同步符号，以免多出一个逗号时会把if语句忽略了)



预测解析的错误恢复

□ 同步记号集合的选择

- 把FOLLOW(A)的所有终结符放入非终结符A的同步记号集合
- 把高层构造的首终结符加到低层构造的同步记号集中
- 把FIRST(A)的终结符加入A的同步记号集合
- 如果出错时**栈顶**是存在有**产生空串选择的非终结符**，则可以使用其推出空串的产生式选择



错误恢复举例

例题7-3 栈顶为 T' ，面临id时出错

非终结符	输入符号			
	id	+	*	...
E	$E \rightarrow TE'$			
E'		$E' \rightarrow +TE'$		
T	$T \rightarrow FT'$			
T'	出错	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	
...				



错误恢复举例

例题7-3 栈顶为 T' ，面临 id 时出错

非终结符	输入符号			
	id	$+$	$*$	$\dots$
E	$E \rightarrow TE'$			
E'		$E' \rightarrow +TE'$		
T	$T \rightarrow FT'$			
T'	出错 用 $T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$	
$\dots$				



预测解析的错误恢复

□ 同步记号集合的选择

- 把FOLLOW(A)的所有终结符放入非终结符A的同步记号集合
- 把高层构造的首终结符加到低层构造的同步记号集中
- 把FIRST(A)的终结符加入A的同步记号集合
- 如果出错时栈顶是存在有产生空串选择的非终结符，则可以使用其推出空串的产生式选择
- 如果终结符在栈顶而不能匹配，弹出此终结符



中国科学技术大学
University of Science and Technology of China

L属性定义的自上而下计算

- 翻译方案
- 预测翻译器的设计
- 用综合属性代替继承属性



翻译方案—内嵌无信息传播的语义动作

例题7-4 把有加和减的中缀表达式翻译成后缀表达式

如果输入是 $8+5-2$ ，则输出是 $8\ 5\ +\ 2\ -$

$$E \rightarrow T R$$
$$R \rightarrow \text{addop } T \{\text{print}(\text{addop.lexeme})\} R_1 \mid \varepsilon$$
$$T \rightarrow \text{num } \{\text{print}(\text{num.val})\}$$
$$E \Rightarrow T R \Rightarrow \text{num } \{\text{print}(8)\} R$$
$$\Rightarrow \text{num } \{\text{print}(8)\} \text{addop } T \{\text{print}(+)\} R$$
$$\Rightarrow \text{num } \{\text{print}(8)\} \text{addop num} \{\text{print}(5)\} \{\text{print}(+)\} R$$
$$\dots \{\text{print}(8)\} \{\text{print}(5)\} \{\text{print}(+)\} \text{addop } T \{\text{print}(-)\} R$$
$$\dots \{\text{print}(8)\} \{\text{print}(5)\} \{\text{print}(+)\} \{\text{print}(2)\} \{\text{print}(-)\}$$

内嵌动作不涉及
属性信息传播



L属性定义及其翻译方案

例题7-5 公式排版：EQN程序示例 $E \sub 1 .val$ $\boxed{E} \boxed{1} \boxed{.val}$
 ps -point size (继承属性); ht -height(综合属性)

产生式	语义规则
$S \rightarrow B$	$B.ps = 10; S.ht = B.ht$
$B \rightarrow B_1 B_2$	$B_1.ps = B.ps; B_2.ps = B.ps;$ $B.ht = \max(B_1.ht, B_2.ht)$
$B \rightarrow B_1 \sub B_2$	$B_1.ps = B.ps; B_2.ps = \text{shrink}(B.ps);$ $B.ht = \text{disp}(B_1.ht, B_2.ht)$
$B \rightarrow \text{text}$	$B.ht = \text{text.h} \times B.ps$



L属性定义及其翻译方案

$S \rightarrow \{B.ps = 10\}$
 $B \{S.ht = B.ht\}$
 $B \rightarrow \{B_1.ps = B.ps\}$
 $B_1 \{B_2.ps = B.ps\}$
 $B_2 \{B.ht = \max(B_1.ht, B_2.ht)\}$
 $B \rightarrow \{B_1.ps = B.ps\}$
 B_1
 $\text{sub} \{B_2.ps = \text{shrink}(B.ps)\}$
 $B_2 \{B.ht = \text{disp}(B_1.ht, B_2.ht)\}$
 $B \rightarrow \text{text} \{B.ht = \text{text.h} \times B.ps\}$

产生式	语义规则
$S \rightarrow B$	$B.ps = 10; S.ht = B.ht$
$B \rightarrow B_1 B_2$	$B_1.ps = B.ps; B_2.ps = B.ps;$ $B.ht = \max(B_1.ht, B_2.ht)$
$B \rightarrow B_1 \text{ sub } B_2$	$B_1.ps = B.ps; B_2.ps = \text{shrink}(B.ps);$ $B.ht = \text{disp}(B_1.ht, B_2.ht)$
$B \rightarrow \text{text}$	$B.ht = \text{text.h} \times B.ps$

属性信息从左向右流动

- 产生式右部符号的继承属性必须在解析该符号之前计算
- 语义动作不能引用其右边符号的综合属性
- 产生式左部非终结符的综合属性只能在其依赖的所有属性都计算完后才能计算



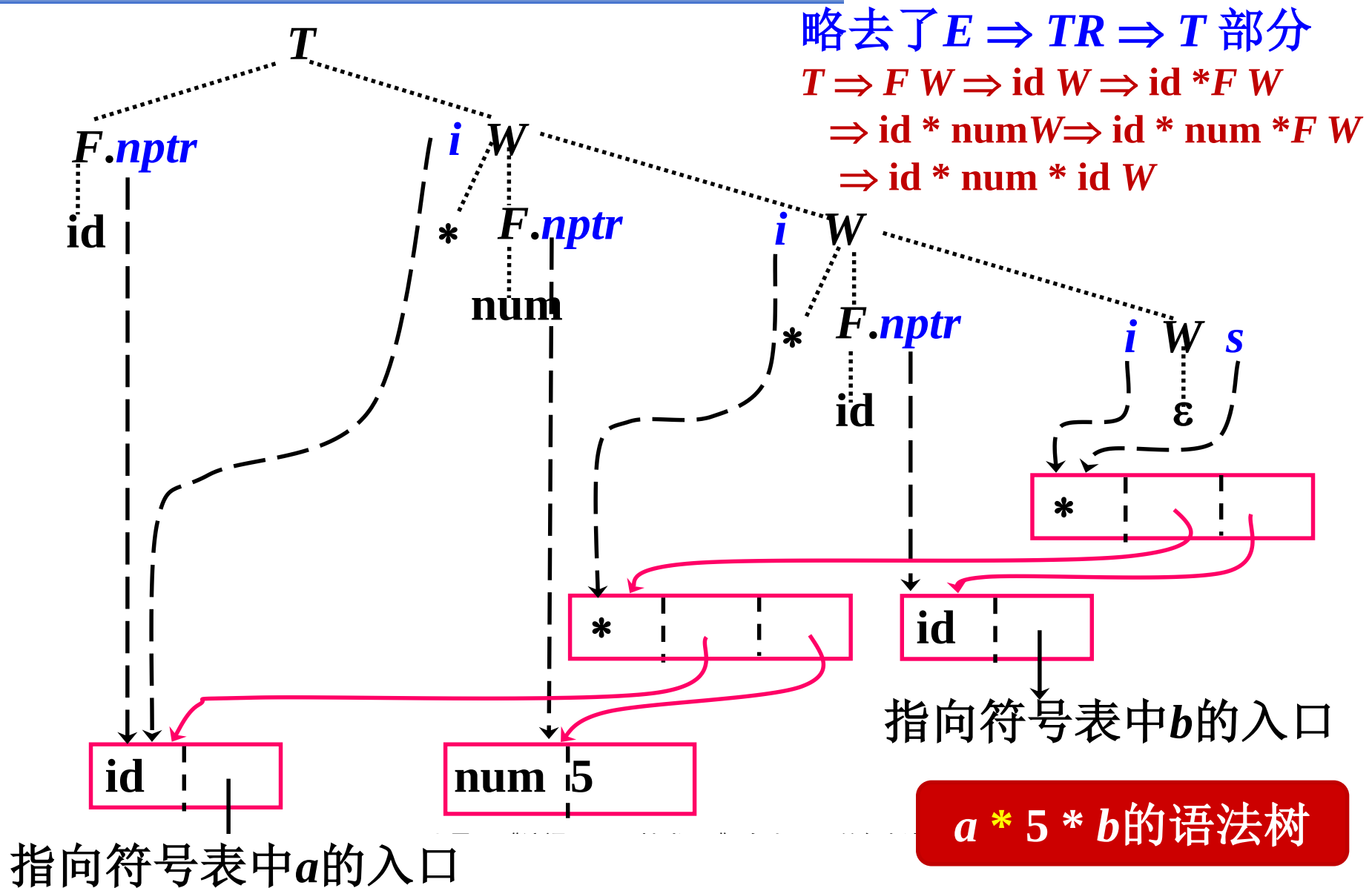
左递归的消除引起继承属性

表达式语言的 LL 文法

产生式	语义规则
$E \rightarrow T R$	$R.i = T.nptr ; E.nptr = R.s$
$R \rightarrow + T R_1$	$R_1.i = mkNode ('+', R.i, T.nptr); R.s = R_1.s$
$R \rightarrow \varepsilon$	$R.s = R.i$
$T \rightarrow F W$	$W.i = F.nptr ; T.nptr = W.s$
$W \rightarrow * F W_1$	$W_1.i = mkNode ('*', W.i, F.nptr); W.s = W_1.s$
$W \rightarrow \varepsilon$	$W.s = W.i$
...	...

产生式	语义规则
$E \rightarrow E_1 + T$	$E.nptr = mkNode('+', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow T_1 * F$	$T.nptr = mkNode('*', T_1.nptr, F.nptr)$
$T \rightarrow F$	$T.nptr = F.nptr$
$F \rightarrow (E)$	$F.nptr = E.nptr$
$F \rightarrow id$	$F.nptr = mkLeaf(id, id.entry)$
$F \rightarrow num$	$F.nptr = mkLeaf(num, num.val)$

语法树的构造 (LL文法)





消除左递归的语法树构造翻译方案

$E \rightarrow T \quad \{R.i = T.nptr\} \quad T + T + T + \dots$
 $R \quad \{E.nptr = R.s\}$
 $R \rightarrow +$
 $T \quad \{R_1.i = mkNode('+', R.i, T.nptr) \}$
 $R_1 \quad \{R.s = R_1.s\}$
 $R \rightarrow \varepsilon \quad \{R.s = R.i\}$
 $T \rightarrow F \quad \{W.i = F.nptr\}$
 $W \quad \{T.nptr = W.s\}$
 $W \rightarrow *$
 $F \quad \{W_1.i = mkNode('*', W.i, F.nptr) \}$
 $W_1 \quad \{W.s = W_1.s\}$
 $W \rightarrow \varepsilon \quad \{W.s = W.i\}$

产生式	语义规则
$E \rightarrow E_1 + T$	$E.nptr = mkNode('+', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow T_1 * F$	$T.nptr = mkNode('*', T_1.nptr, F.nptr)$
$T \rightarrow F$	$T.nptr = F.nptr$
$F \rightarrow (E)$	$F.nptr = E.nptr$
$F \rightarrow id$	$F.nptr = mkLeaf(id, id.entry)$
$F \rightarrow num$	$F.nptr = mkLeaf(num, num.val)$

继承属性的计算内嵌在产生式
右部的某文法符号之前, 表示
在分析该文法符号之前计算

F 的产生式部分不再给出



中国科学技术大学
University of Science and Technology of China

L属性定义的自上而下计算

- 翻译方案
- 预测翻译器的设计
- 用综合属性代替继承属性



预测翻译器的设计

方法：将预测解析器的构造方法推广到翻译方案的实现（*LL*文法）

产生式 $R \rightarrow +TR \mid \epsilon$ 的分析过程

```
void R( ) {  
    if (lookahead == '+' ) {           // First(+TR)  
        match ( '+' ); T( ); R( );  
    }  
    else if (lookahead == ')' || lookahead == '$' ) ; // Follow(R)  
    else error( );  
}
```



预测翻译器的设计

```
syntaxTreeNode * R (syntaxTreeNode * i) {  
    //继承属性作为参数, 综合属性作为返回值  
    syntaxTreeNode *nptr, *i1, *s1, *s;  
    char addoplexeme;  
  
    if (lookahead == '+' ) {  
        addoplexeme = lexval;  
        match('+' ); nptr = T( );  
        i1 = mkNode(addoplexeme, i , nptr);  
        s1 = R (i1); s = s1;  
    }  
    else if (lookahead == ')' || lookahead == '$') s = i;  
    else error( );  
    return s;  
}
```

```
void R( ) {  
    if (lookahead == '+' ) {  
        match ( '+' ); T( ); R( );  
    }  
    else if (lookahead == ')' || lookahead == '$' ) ;  
    else error( );  
}
```

R : i, s
T : nptr
+ : addoplexeme



中国科学技术大学
University of Science and Technology of China

L属性定义的自上而下计算

- ☐ 翻译方案
- ☐ 预测翻译器的设计
- ☐ 用综合属性代替继承属性



非L属性定义

例题7-6 Pascal语言的变量声明，如 **m, n : integer**

$D \rightarrow L : T$ $L.in = T.type$

$L \rightarrow L_1, id \mid id$ $L_1.in = L.in, \dots$

$T \rightarrow integer \mid char$ $T.type = \dots$

该语法制导定义**非L属性定义**

信息从右向左流，归约从左向右，两者不一致



非L属性定义：改写文法

例题7-6 Pascal的声明，如 $m, n : \text{integer}$

$D \rightarrow L : T$ $L.in = T.type$ (非L属性定义：属性信息从右流向左)

$L \rightarrow L_1, \text{id} \mid \text{id}$ $L_1.in = L.in, \dots$

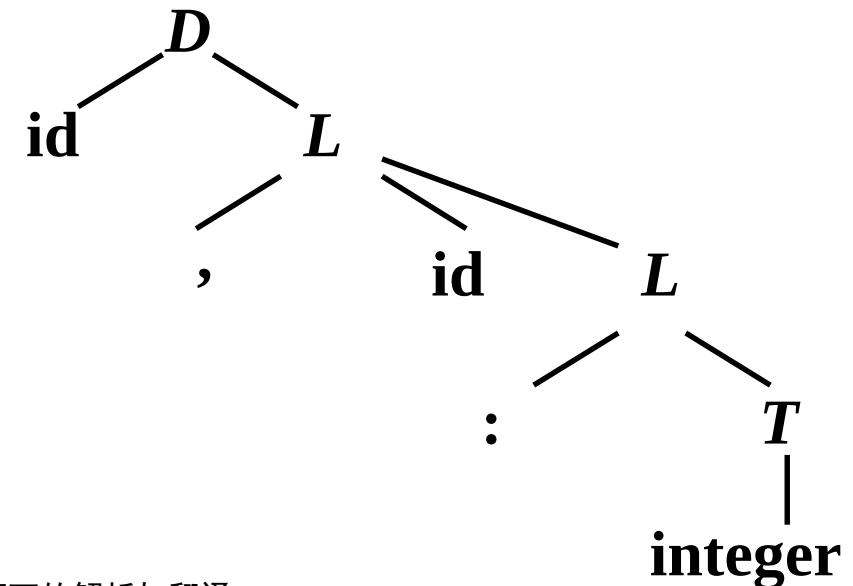
$T \rightarrow \text{integer} \mid \text{char}$ $T.type = \dots$

等所需信息获得后再归约，改成从右向左归约

$D \rightarrow \text{id } L$ (S属性定义)

$L \rightarrow , \text{id } L \mid : T$

$T \rightarrow \text{integer} \mid \text{char}$





用综合属性代替继承属性

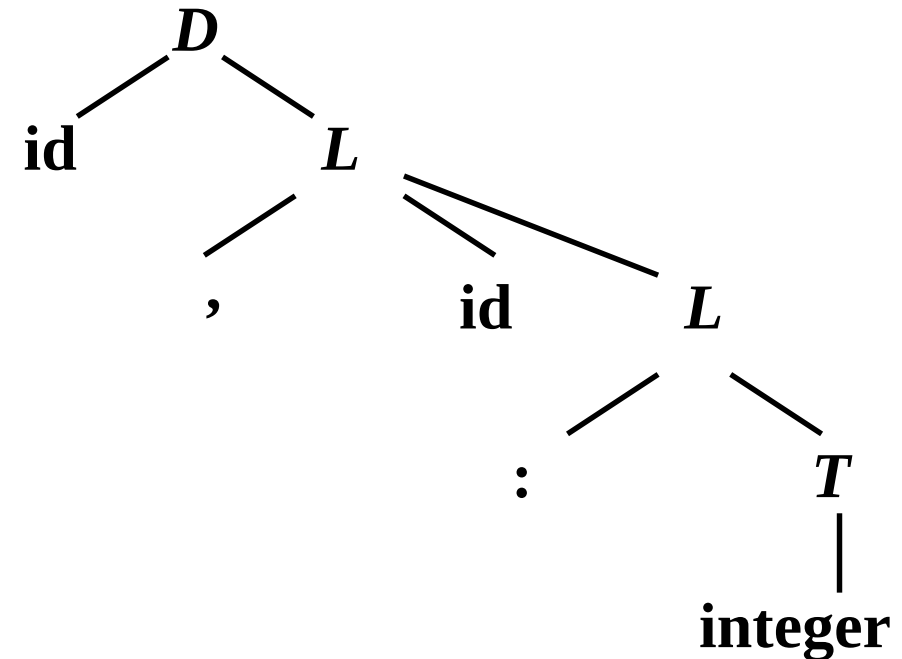
$D \rightarrow \text{id } L \quad \{ \text{addtype}(\text{id. entry}, L.\text{type}); \}$

$L \rightarrow , \text{id } L_1 \quad \{ L.\text{type} = L_1.\text{Type};$
 $\quad \text{addtype}(\text{id. entry}, L_1.\text{type}); \}$

$L \rightarrow : T \quad \{ L.\text{type} = T.\text{type}; \}$

$T \rightarrow \text{integer} \quad \{ T.\text{type} = \text{integer}; \}$

$T \rightarrow \text{real} \quad \{ T.\text{type} = \text{real}; \}$





ANTLR 的原理

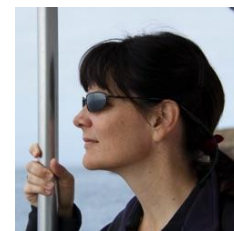
- ANTLR3: $LL(*)$ [[PLDI2011](#)]
- ANTLR4:
Adaptive $LL(*)$ [[OOPSLA2014](#)]



[Terence Parr](#)
Univ. of San Fran.



[Sam Harwell](#)
Microsoft



[Kathleen Fisher](#)
Tufts University



ANTLR的引入

使用**向前看正规式**区分不同产生式选择，提供近似确定性分析

□ LL(*) – ANTLR 3.3 [PLDI 2011]

- 引入**语法谓词**、**语义谓词**支持任意的向前看(lookahead)
- 文法不能是左递归的（但可以自动消除直接左递归）
- **不足**：LL(*)文法条件静态不可判定，故有时找不到向前看正规式来区分

□ Adaptive LL(*) – ANTLR 4 [OOPSLA 2014]

- 引入语义谓词和动作产生式，支持所有的LL(*)文法
- 在决策点发起多个子解析器：**并发解析**
- **记忆解析结果，增量动态构建DFA**
- 使用**GSS(graph-structured stack)**避免冗余计算



ANTLR 3: LL(*)

□ 谓词文法 $G = (N, T, P, S, \Pi, \mathcal{M})$

■ N :非终结符集合

■ T :终结符集合

■ P :产生式集合

■ $S \in N$:开始符

■ Π :无副作用的
语义谓词

■ $\mathcal{M}$:动作集合
mutators 状态修改器

$A \in N$

$a \in T$

$X \in (N \cup T)$

$\alpha, \beta, \delta \in X^*$

$u, x, y, w \in T^*$

$w_r \in T^*$

ϵ

$\pi \in \Pi$

$\mu \in \mathcal{M}$

$\lambda \in (N \cup \Pi \cup \mathcal{M})$

$\vec{\lambda} = \lambda_1.. \lambda_n$

Production Rules:

$A \rightarrow \alpha_i$ i^{th} context-free production of A

$A \rightarrow (A'_i) \Rightarrow \alpha_i$ i^{th} production predicated on syntax A'_i

$A \rightarrow \{\pi_i\} ? \alpha_i$ i^{th} production predicated on semantics

$A \rightarrow \{\mu_i\}$ i^{th} production with mutator

Nonterminal

Terminal

Grammar symbol

Sequence of grammar symbols

Sequence of terminals

Remaining input terminals

Empty string

Predicate in host language

Action in host language

Reduction label

Sequence of reduction labels

[[PLDI2011](#)] Terence Parr, Kathleen Fisher. LL(*): The Foundation of the ANTLR Parser Generator.



LL(*)的产生式形式以及二义性消除

□ 产生式的形式

■ 标准的产生式: $A \rightarrow \alpha_i$

■ 含语法谓词的产生式: $A \rightarrow (A'_i) \Rightarrow \alpha_i$

仅当当前输入也匹配由 A'_i 描述的语法时, A 展开成 α_i

□ 语法谓词(A'_i) $\Rightarrow$ 可以替换为语义谓词 $\{\text{synpred}(A'_i)\}$?

■ 含语义谓词的产生式: $A \rightarrow \{\pi_i\}? \alpha_i$

仅当到目前所构造的状态满足谓词 π_i 时, A 展开成 α_i

■ 动作: $A \rightarrow \{\mu_i\}$ 根据动作 μ_i 更新状态

□ 二义性的消除

■ 指定语义谓词来消除歧义

■ 冲突时选择位于文法中靠前的产生式规则



LL(*)谓词文法的最左推导

□ 谓词文法的最左推导规则

■ 判断形式(judgement form)

The judgment form $(S, \alpha) \xRightarrow{\lambda} (S', \beta)$, may be read: “In machine state S , grammar sequence α reduces in one step to modified state S' and grammar sequence β while emitting trace λ .”

■ 推导规则

$$\begin{array}{ll} \text{Prod} \frac{A \rightarrow \alpha}{(S, uA\delta) \Rightarrow (S, u\alpha\delta)} & \text{Action} \frac{A \rightarrow \{\mu\}}{(S, uA\delta) \xRightarrow{\mu} (\mu(S), u\delta)} \\ \text{Sem} \frac{\pi_i(S) \quad A \rightarrow \{\pi_i\} ? \alpha_i}{(S, uA\delta) \xRightarrow{\pi_i} (S, u\alpha_i\delta)} & \text{Syn} \frac{\begin{array}{l} (S, A'_i) \Rightarrow^* (S', w) \\ w \preceq w_r \\ A \rightarrow (A'_i) \Rightarrow \alpha_i \end{array}}{(S, uA\delta) \xRightarrow{A'_i} (S, u\alpha_i\delta)} \end{array}$$

在当前状态仅当由 A'_i 推导出的串 w 是剩余输入串 w_r 的前缀

$$\text{Closure} \frac{(S, \alpha) \xRightarrow{\lambda} (S, \alpha'), (S, \alpha') \xRightarrow{\vec{\lambda}}^* (S, \beta)}{(S, \alpha) \xRightarrow{\lambda \vec{\lambda}}^* (S, \beta)}$$



LL(*)向前看DFA

□ 文法到增强转换网络ATN的构造

输入文法的元素	对应的 ATN 状态转换
$A \rightarrow \alpha_i$	$p_A \xrightarrow{\epsilon} p_{A,i} \xrightarrow{\epsilon} \boxed{\alpha_i} \xrightarrow{\epsilon} p'_A$
$A \rightarrow \{\pi_i\}?\alpha_i$	$p_A \xrightarrow{\epsilon} p_{A,i} \xrightarrow{\pi_i} \boxed{\alpha_i} \xrightarrow{\epsilon} p'_A$
$A \rightarrow \{\mu_i\}$	$p_A \xrightarrow{\epsilon} p_{A,i} \xrightarrow{\mu_i} p'_A$
$A \rightarrow \epsilon$	$p_A \xrightarrow{\epsilon} p_{A,i} \xrightarrow{\epsilon} p'_A$
$\boxed{\alpha_i} = X_1X_2 \dots X_m$	$p_0 \xrightarrow{X_1} p_1 \xrightarrow{X_2} \dots \xrightarrow{X_m} p_m$

■ ATN中非终结符边(边标记为A):

- 对A分析函数的调用, 控制权转移到A的子自动机

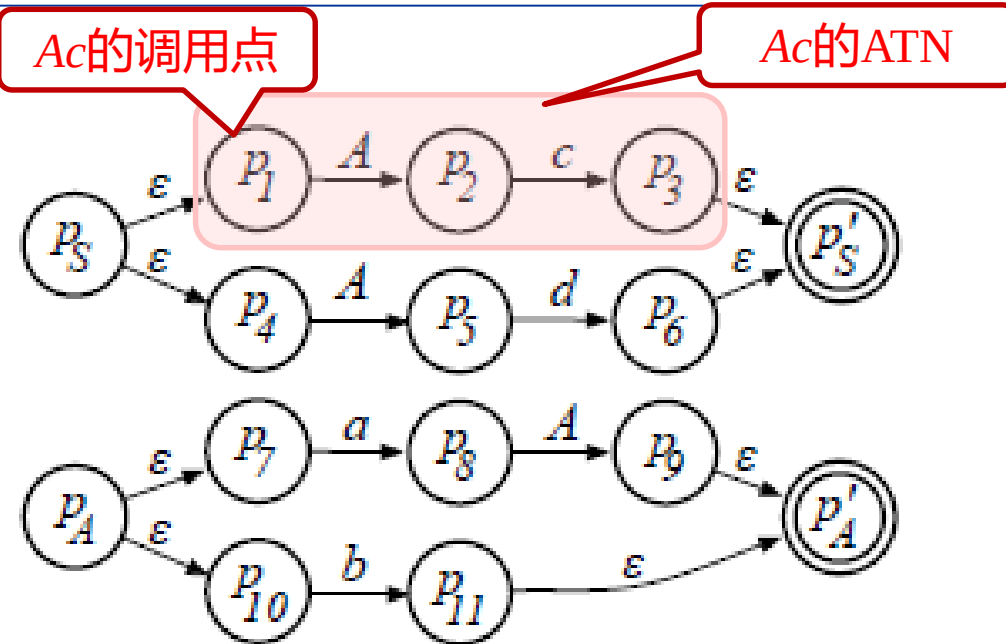
□ 向前看DFA的状态变化

■ DFA状态: 一组ATN格局

$$\frac{p \xrightarrow{a} q}{(\mathbb{S}, p, aw) \mapsto (\mathbb{S}, q, w)}$$

$$\frac{\pi_i(\mathbb{S}) \quad p \xrightarrow{\pi_i} f_i}{(\mathbb{S}, p, w) \xrightarrow{\pi_i} (\mathbb{S}, f_i, w)}$$

$$\frac{(\mathbb{S}, f_i, w)}{\text{Accept, 预测产生式 } i}$$



$$P = \{S \rightarrow \text{Ac} \mid Ad, A \rightarrow aA \mid b\}$$



ANTLR4的Adaptive LL(*)

□ LL(*)的主要问题

- 静态不可判定
- 文法分析有时找不到能区分不同产生式选择的正规式
- 回溯决策也不能检测如下的二义性： $A \rightarrow \alpha|\alpha$
如果 α 是使得 $\alpha|\alpha$ 非LL(*)的文法符号序列

□ Adaptive LL(*)，即 ALL(*)

- **动态分析**：将文法分析移到parse-time
 - 避免LL(*)静态文法分析的不可判定性
 - 可以为任何非左递归上下文无关文法产生正确的解析器



ALL(*)谓词文法

□ 文法 $G = (N, T, P, S, \Pi, \mathcal{M})$

- N : 非终结符集合
- T : 终结符集合
- P : 产生式集合
- $S \in N$: 开始符
- Π : 无副作用语义谓词
- $\mathcal{M}$: 动作集合

$A \in N$	Nonterminal
$a, b, c, d \in T$	Terminal
$X \in (N \cup T)$	Production element
$\alpha, \beta, \delta \in X^*$	Sequence of grammar symbols
$u, v, w, x, y \in T^*$	Sequence of terminals
ϵ	Empty string
$\$$	End of file "symbol"
$\pi \in \Pi$	Predicate in host language
$\mu \in \mathcal{M}$	Action in host language
$\lambda \in (N \cup \Pi \cup \mathcal{M})$	Reduction label
$\vec{\lambda} = \lambda_1.. \lambda_n$	Sequence of reduction labels

Production Rules:

$A \rightarrow \alpha_i$	i^{th} context-free production of A
$A \rightarrow \{\pi_i\}? \alpha_i$	i^{th} production predicated on semantics
$A \rightarrow \{\mu_i\}$	i^{th} production with mutator

□ 最左推导规则

$$\begin{array}{c} \text{Prod} \frac{A \rightarrow \alpha}{(\mathbb{S}, uA\delta) \Rightarrow (\mathbb{S}, u\alpha\delta)} \\ \text{Sem} \frac{\pi(\mathbb{S}) \quad A \rightarrow \{\pi\}? \alpha}{(\mathbb{S}, uA\delta) \Rightarrow (\mathbb{S}, u\alpha\delta)} \quad \text{Action} \frac{A \rightarrow \{\mu\}}{(\mathbb{S}, uA\delta) \Rightarrow (\mu(\mathbb{S}), u\delta)} \\ \text{Closure} \frac{(\mathbb{S}, \alpha) \Rightarrow (\mathbb{S}', \alpha'), (\mathbb{S}', \alpha') \Rightarrow^* (\mathbb{S}'', \beta)}{(\mathbb{S}, \alpha) \Rightarrow^* (\mathbb{S}'', \beta)} \end{array}$$



ALL(*)

□ 预测机制

- 在决策点，为每个产生式选择发起一个子解析器，各子解析器可并发探索
- 调用栈敏感的预测

$S \rightarrow xB \mid yC$ $B \rightarrow Aa$ $C \rightarrow Aba$ $A \rightarrow b \mid \varepsilon$

- 无解析器栈（强LL解析器）：在A的决策点面临ba时，A的两个选择都可以——预测冲突
- 有解析器栈：B调用A时预测使用 $A \rightarrow b$ 展开，C调用A时预测使用 $A \rightarrow \varepsilon$ 展开

- 使用 **图结构栈(GSS)**避免冗余计算

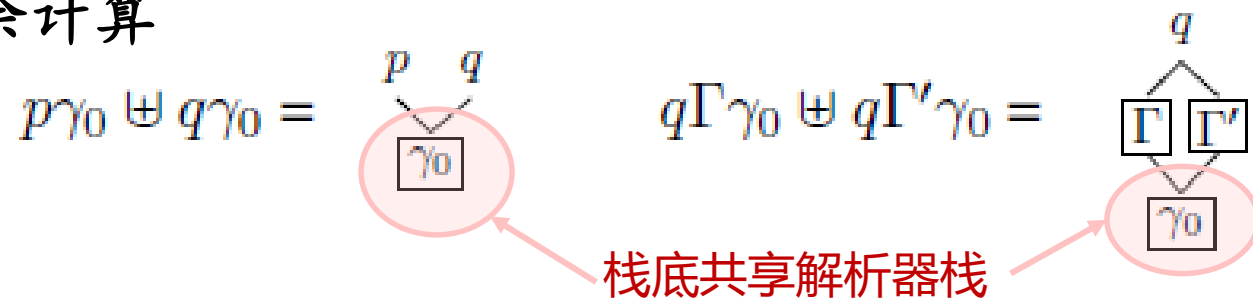
□ 记忆分析结果

- **增量动态**构建DFA

建立向前看短语到预测的产生式编号的映射

ALL(*) 解析器分析的复杂度 $O(n^4)$

张昱：《编译原理和技术(H)》自上而下的解析与翻译





ANTLR应用举例

```
grammar Expr;
```

```
@header {  
package tools;  
import java.util.*;  
}
```

```
@parser::members {  
/** "memory" for our calculator;  
    variable/value pairs go here */  
Map<String, Integer> memory =  
    new HashMap<String, Integer>();  
int eval(int left, int op, int right) {  
    switch ( op ) {  
        case MUL : return left * right;  
        case ADD : return left + right;  
        .....  
    }  
    return 0;  
}  
}
```

输入程序及
其解析结果

```
a=3 +4 ;  
b=a+4 ;
```

```
program: stat+ ;  
stat:   e NEWLINE      {System.out.println($e.v);}  
      | ID '=' e NEWLINE {memory.put($ID.text, $e.v); System.out.println($ID.text+"="+$e.v);}  
      | NEWLINE  
      ;  
e returns [int v] : a=e op=('*'|'/') b=e {$v = eval($a.v, $op.type, $b.v);}  
                  | a=e op=('+'|'-') b=e {$v = eval($a.v, $op.type, $b.v);}  
                  | INT                {$v = $INT.int;}  
                  | ID { String id = $ID.text; $v = memory.containsKey(id) ? memory.get(id) : 0; }  
                  | '(' e ')'          {$v = $e.v;}  
                  ;  
MUL : '*' ;  
DIV : '/' ;  
ADD : '+' ;  
SUB : '-' ;  
ID  : [a-zA-Z]+ ; // match identifiers  
INT : [0-9]+ ;    // match integers  
NEWLINE: '\r'? '\n' ; // return newlines to parser (is end-statement signal)  
WS : [ \t]+ -> skip ; // toss out whitespace
```

```
a=7  
b=11  
(program (stat a = (e (e 3) + (e 4)) ; \n) (stat b = (e (e a) + (e 4)) ;
```



ANTLR应用：文法可复用

□ 将文法分解成多个可复用的块

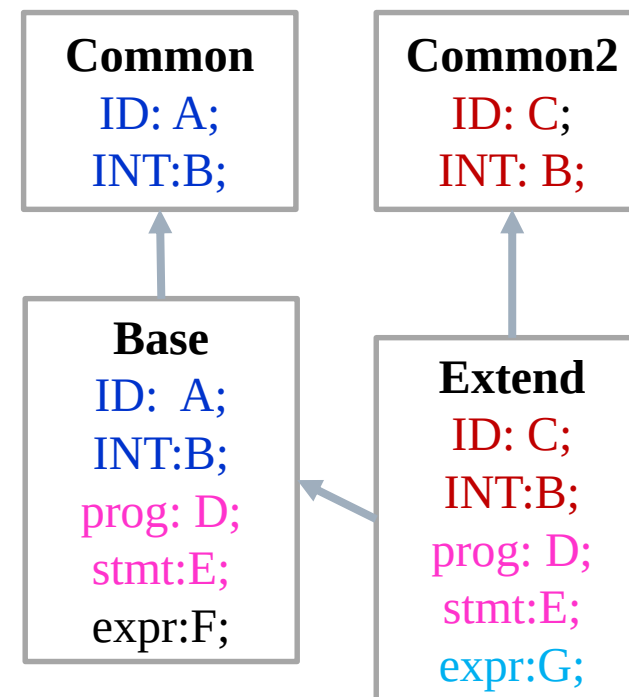
■ 通过import导入的文法类似于面向对象语言中的超类

```
lexer grammar Common;  
ID : [a-zA-Z_][a-zA-Z_0-9]* ;  
INT : [0-9]+ ;  
WS : [ \t\r\n]+ -> skip ;
```

```
grammar Base;  
import Common;  
prog : stmt* ;  
stmt : expr ';' ;  
expr : ID ;
```

```
lexer grammar Common2;  
ID : [a-zA-Z][a-zA-Z_0-9]* ;  
INT : [0-9]+ ;  
WS : [ \t\r\n]+ -> skip ;
```

```
grammar Extend;  
import Common2,Base;  
  
// 扩展表达式规则  
expr : ID | INT ;
```



<https://github.com/antlr/antlr4/blob/master/doc/grammars.md>